Kubernetes and Minikube.

What It Actually Is

kubectl is **just a client tool**.

It does NOT run Kubernetes.
It does NOT create clusters.
It simply talks to a Kubernetes API Server.

What It Actually Is

Minikube is a **local Kubernetes cluster**.

It runs:

- 1 control plane node
- 1 worker node (usually same VM/container)
- On your laptop

Install the Docker and start the docker.

```
-rw-r--r--.  1 ec2-user docker   134891912 Feb 26 07:57 minikube-linux-amd64
[ec2-user@ip-172-31-90-195 ~]$ systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: disabled)
     Active: active (running) since Thu 2026-02-26 10:21:46 UTC; 4min 31s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Process: 2903 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
    Process: 2904 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)
   Main PID: 2905 (dockerd)
      Tasks: 12
     Memory: 1.4G
        CPU: 13.213s
     CGroup: /system.slice/docker.service
             └─2905 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

Feb 26 10:21:44 ip-172-31-90-195.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Feb 26 10:21:45 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:45.3028017512" level=info msg="Starting up"
Feb 26 10:21:45 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:45.3813886832" level=info msg="[graphdriver] using prior storage driver: overlay2"
Feb 26 10:21:45 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:45.3835790102" level=info msg="Loading containers: start."
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:46.0722569362" level=info msg="Default bridge (docker0) is assigned with an IP add
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:46.2271580152" level=info msg="Loading containers: done."
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:46.2542558002" level=info msg="Docker daemon" commit=d334795 containerd-snapshotte
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:46.2543392882" level=info msg="Daemon has completed initialization"
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal dockerd[2905]: time="2026-02-26T10:21:46.2871011112" level=info msg="API listen on /run/docker.sock"

i-0fd3ba4cefceadbdd (minikube)
PublicIPs: 13.220.194.85   PrivateIPs: 172.31.90.195
```

After the installation of all the packages and services we have to start the docker (docker is important for minikube).we have to give user level permission to the docker to execute seamlessly.

```
total 131748
drwx------.  4 ec2-user ec2-user      140 Feb 26 08:03 .
drwxr-xr-x.  3 root     root           22 Feb 26 07:52 ..
-rw-------.  1 ec2-user docker        699 Feb 26 08:03 .bash_history
-rw-r--r--.  1 ec2-user ec2-user       18 Jan 28  2023 .bash_logout
-rw-r--r--.  1 ec2-user ec2-user      141 Jan 28  2023 .bash_profile
-rw-r--r--.  1 ec2-user ec2-user      492 Jan 28  2023 .bashrc
drwxr-xr-x. 10 ec2-user docker        117 Feb 26 10:21 .minikube
drwx------.  2 ec2-user ec2-user       29 Feb 26 07:52 .ssh
-rw-r--r--.  1 ec2-user docker   134891912 Feb 26 07:57 minikube-linux-amd64
[ec2-user@ip-172-31-90-195 ~]$ systemctl status docker
```

Then we have to install the minikube in the machine.

```
Feb 26 10:21:46 ip-172-31-90-195.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.

[ec2-user@ip-172-31-90-195 ~]$ minikube start --driver=docker
* minikube v1.38.1 on Amazon 2023.10.20260216
X0226 10:28:23.389812    3440 start.go:846] api.Load failed for minikube: filestore "minikube": Docker machine "minikube" does not exist. Use "docker-machine ls" to li
st machines. Use "docker-machine create" to add a new one.
X0226 10:28:23.389953    3440 start.go:846] api.Load failed for minikube: filestore "minikube": Docker machine "minikube" does not exist. Use "docker-machine ls" to li
st machines. Use "docker-machine create" to add a new one.
* Using the docker driver based on existing profile

X The requested memory allocation of 3072MiB does not leave room for system overhead (total system memory: 3819MiB). You may face stability issues.
* Suggestion: Start minikube with less memory allocated: 'minikube start --memory=3072mb'

* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
[ec2-user@ip-172-31-90-195 ~]$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   73s   v1.35.1
[ec2-user@ip-172-31-90-195 ~]$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Then we have to install and check the kubectl has been installed or not.

```
[ec2-user@ip-172-31-90-65 ~]$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 57224k 100 57224k    0     0 474.8M     0  --:--:-- --:--:-- --:--:-- 477.6M
[ec2-user@ip-172-31-90-65 ~]$ ls
kubectl
[ec2-user@ip-172-31-90-65 ~]$ chmod +x kubectl
sudo mv kubectl /usr/local/bin/
[ec2-user@ip-172-31-90-65 ~]$ ls
[ec2-user@ip-172-31-90-65 ~]$ kubectl version -client
error: extra arguments: [-client]
[ec2-user@ip-172-31-90-65 ~]$ kubectl version
Client Version: v1.35.1
Kustomize Version: v5.7.1
Server Version: v1.35.1
[ec2-user@ip-172-31-90-65 ~]$
```

This is the normal pod.yaml file.

```
  GNU nano 8.3                                   pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mynewpod
spec:
  containers:
    - name: mynewimage
      image: httpd
    - name: mysecondimage
      image: nginx

                                  [ Read 10 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-B Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^B Where Was    M-F Next
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195

Using the **kubectl apply -f<filename>.yaml** command we can create an pod , namespaces .

```
[ec2-user@ip-172-31-90-65 ~]$ kubectl apply -f pod.yaml
pod/mypod created
[ec2-user@ip-172-31-90-65 ~]$ kubectl get pods
NAME        READY   STATUS             RESTARTS      AGE
mynewpod    1/2     CrashLoopBackOff   5 (80s ago)   4m28s
mypod       1/1     Running            0             8s
```

The deployment.yaml file.

```
GNU nano 8.3                                   Deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: mycontainer
          image: nginx
          ports:
            - containerPort: 80

                                   [ Read 20 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute     ^C Location    M-U Undo      M-A Set Mark   M-] To Bracket  M-B Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go To Line  M-E Redo      M-6 Copy       ^B Where Was    M-D Next
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195

Using “kuberctl apply -f <Deployment.yaml> command we can create an pod.

```
[ec2-user@ip-172-31-90-195 ~]$ kubectl get pod
NAME                            READY   STATUS    RESTARTS        AGE
mydeployment-5f7f5757b6-2ssgx   1/1     Running   0               33s
mydeployment-5f7f5757b6-9dwxb   1/1     Running   0               33s
mydeployment-5f7f5757b6-c6d2d   1/1     Running   0               32s
mydeployment-5f7f5757b6-kh6sz   1/1     Running   0               33s
mydeployment-5f7f5757b6-zcfgl   1/1     Running   0               33s
```

The developmentnamespace.yaml file.

```
[ec2-user@ip-172-31-90-65 ~]$ vi developementnamespace.yaml
[ec2-user@ip-172-31-90-65 ~]$ cat developementnamespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: developement
  labels:
    name: developement

[ec2-user@ip-172-31-90-65 ~]$
```

Using this I have created one namespace .

```
[ec2-user@ip-172-31-90-195 ~]$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   32m
development       Active   14s
kube-node-lease   Active   32m
kube-public       Active   32m
kube-system       Active   32m
[ec2-user@ip-172-31-90-195 ~]$
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195

When we crating an pod in the namespace level means we have to call the namespace to find the particular namespace.

Kubectl get pods -n<namespace name>

```
[ec2-user@ip-172-31-90-195 ~]$ kubectl get pod -n development
NAME                              READY   STATUS    RESTARTS   AGE
mydeployment-5f7f5757b6-6vw9q     1/1     Running   0          4m9s
mydeployment-5f7f5757b6-ccdq7     1/1     Running   0          4m9s
mydeployment-5f7f5757b6-dqt4d     1/1     Running   0          4m9s
mydeployment-5f7f5757b6-ffzx7     1/1     Running   0          4m9s
mydeployment-5f7f5757b6-rkjt9     1/1     Running   0          4m9s
[ec2-user@ip-172-31-90-195 ~]$
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195

The Replicaside.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

```
                                                    [ Read 19 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute     ^C Location    M-U Undo      M-A Set Mark   M-] To Bracket  M-B Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go To Line  M-E Redo      M-6 Copy       ^B Where Was    M-F Next
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195

Using “kuberctl get rs”

```
[ec2-user@ip-172-31-90-195 ~]$ nano replicaset.yaml
[ec2-user@ip-172-31-90-195 ~]$ kubectl apply -f replicaset.yaml
replicaset.apps/nginx-rs created
[ec2-user@ip-172-31-90-195 ~]$ kubectl get rs
NAME                        DESIRED   CURRENT   READY   AGE
mydeployment-5f7f5757b6     5         5         5       21m
nginx-rs                    3         3         3       13s
[ec2-user@ip-172-31-90-195 ~]$
```

i-0fd3ba4cefceadbdd (minikube)

PublicIPs: 13.220.194.85 PrivateIPs: 172.31.90.195